M-Agent Chat API Reference

Combined from `docs/chat_api/README.md` and `docs/chat_api/testing.http`.

This PDF is intended for software development, integration, and manual API testing.

Contents

M-Agent Chat API Reference

> Status / 状态: aligned with the current implementation in `src/m_agent/api/` Audience / 读者: backend developers, frontend developers, QA, and integration testers Scope / 范围: HTTP JSON APIs, SSE streams, auth, user config, chat runs, dialogue archives, thread memory, and schedules

1. Overview / 总览

This document replaces the old single-file `docs/chat_api.md` and is written against the current FastAPI implementation in:

- `src/m_agent/api/chat_api_web.py`
- `src/m_agent/api/chat_api_runtime.py`
- `src/m_agent/api/chat_api_records.py`

- `src/m_agent/api/user_access.py`
- `src/m_agent/api/chat_dialogue_store.py`
- `src/m_agent/api/schedule_heartbeat.py`

当前 Chat API 不是“每次请求携带完整 config”的模式，而是“服务启动时固定 config，运行时维护 thread session”的模式。

The current Chat API is not a per-request config override service. It uses a startup-fixed runtime config and keeps thread-scoped session state in memory while the server is alive.

1.1 Core runtime model / 核心运行模型

- Think-life is the sole runtime. There is no runtime-mode selector.
- Think-life 是唯一运行时，不存在运行模式切换项。
- `POST /v1/chat/runs` creates an asynchronous chat run.
- `GET /v1/chat/runs/{run_id}/events` is the main real-time event stream for one chat run.
- `GET /v1/chat/runs/{run_id}` returns the final run snapshot.
- `thread_id` identifies a long-lived Think-life thread with a Scene timeline and transaction state.
- When auth is enabled, the server internally scopes thread ids as `username::public_thread_id`.
- Thread memory currently supports two modes:
- `manual` : new chat rounds enter the pending memory buffer and can later be flushed
- `off` : new chat rounds remain available to current chat history, but do not enter the pending memory buffer

1.2 Important separation / 一个重要的接口边界

- `GET /v1/chat/runs/{run_id}/events` is the detailed per-run trace channel.
- `GET /v1/chat/threads/{thread_id}/events` is the thread lifecycle channel.

Do not treat them as interchangeable.

不要把它们当成同一个事件流来消费。

In the current implementation:

- normal user chat trace events belong to the run stream
- thread events mainly cover memory state changes, flush progress, schedule CRUD, and schedule execution

2. Quick Navigation / 快速导航

Group	Endpoints	中文说明	English description
Health	`GET /` `GET /healthz`	服务健康、运行参数、端点清单	Service health, runtime metadata, endpoint map
Auth	`POST /v1/auth/register` `POST /v1/auth/login` `GET /v1/auth/me` `POST /v1/auth/logout`	用户注册、登录、会话信息、登出	Registration, login, session inspection, logout
User config	`GET /v1/users/me/config/schema` `PATCH /v1/users/me/config`	当前用户可编辑配置元数据与更新接口	Editable config metadata and patch API

Group	Endpoints	中文说明	English description
Chat runs	`POST /v1/chat/runs` `GET /v1/chat/runs/{run_id}` `GET /v1/chat/runs/{run_id}/events`	创建对话、获取结果、订阅 run 级事件	Create run, fetch final result, subscribe to run events
Thread events	`GET /v1/chat/threads/{thread_id}/events`	线程级事件流	Thread-level SSE stream
Thread memory	`GET /v1/chat/threads/{thread_id}/memory/state` `POST /v1/chat/threads/{thread_id}/memory/mode` `POST /v1/chat/threads/{thread_id}/memory/flush`	查看线程记忆状态、切换模式、手动 flush	Inspect thread state, switch memory mode, manually flush
Dialogues	`GET /v1/chat/dialogues` `GET /v1/chat/dialogues/{dialogue_id}`	已 flush 的对话归档列表与详情	Flushed dialogue archive list and detail
Schedules	`GET/POST/PATCH/DELETE /v1/chat/threads/{thread_id}/schedules...`	日程提醒查询、创建、更新、取消	Schedule query, create, update, cancel

3. Startup / 启动方式

3.1 Example / 示例

```
$env:PYTHONPATH = "src"
python -m m_agent.api.chat_api `
  --host 127.0.0.1 `
  --port 8777 `
  --config config/agents/chat/chat_controller.yaml `
  --idle-flush-seconds 1800 `
  --history-max-rounds 12 `
  --schedule-beat-seconds 10 `
  --schedule-enqueue-retry-seconds 5 `
  --users-db config/users/users.json `
  --session-ttl-seconds 43200
```

3.2 Startup arguments / 启动参数

Arg	Default	中文说明	English description
`--host`	`127.0.0.1`	绑定地址	Bind host
`--port`	`8777`	绑定端口	Bind port

Arg	Default	中文说明	English description
`--config`	`config/agents/chat/chat_controller.yaml`	启动后固定的 chat config	Startup-fixed chat config
`--idle-flush-seconds`	`1800`	`manual` 模式下 pending buffer 的空闲自动 flush 时间	Idle timeout before pending manual memory is auto-flushed
`--history-max-rounds`	`12`	每个线程在服务内保留的最大轮次数	Max in-memory rounds retained per thread
`--schedule-beat-seconds`	`10`	日程心跳扫描周期	Schedule heartbeat scan interval
`--schedule-enqueue-retry-seconds`	`5`	日程刺激入队失败后的 lease 重试延迟	Lease retry delay after a schedule stimulus fails to enqueue
`--users-db`	`config/users/users.json`	用户数据库路径	User database path
`--session-ttl-seconds`	`43200`	登录会话有效期，单位秒	Session TTL in seconds
`--disable-auth`	off	关闭注册/登录与 Bearer 校验，进入匿名模式	Disable auth and run in anonymous mode
`--debug`	off	打开更详细的服务日志	Enable verbose backend logs

3.3 Built-in docs / 内置文档

- Swagger UI: `http://127.0.0.1:8777/docs`
- OpenAPI JSON: `http://127.0.0.1:8777/openapi.json`

4. Common Conventions / 通用约定

Item	Value / Format	中文说明	English description
Base content type	`application/json`	普通 HTTP 接口收发 JSON	Regular HTTP endpoints use JSON
SSE content type	`text/event-stream`	事件流使用标准 SSE	Event streams use standard SSE
Time format	ISO 8601, usually UTC with `Z`	时间戳通常为 UTC `Z` 格式	Timestamps are usually ISO UTC with `Z`
Auth header	`Authorization: Bearer <token>`	默认会话认证头	Primary auth header
Alternate auth header	`X-Session-Token: <token>`	备用会话认证头	Alternate session header
Error envelope	`{"error": "..."}`	失败时至少包含 `error` 字段	Failures contain at least `error`
Run polling resume	`after_seq` on run SSE, default `0`	run 事件流默认可从头追事件	Run stream can replay from the beginning by default
Thread live tail	`after_seq` on thread SSE, default `-1`	thread 事件流默认从“当前尾部”开始订阅新事件	Thread stream follows only new events by default

Item	Value / Format	中文说明	English description
Config override	request body `config` is rejected	当前服务不支持请求级切换 config	Request-level config override is not supported

4.1 Auth behavior / 鉴权行为

- When auth is enabled, chat APIs require a valid session token.
- When auth is disabled by `--disable-auth`, auth endpoints return `503`, but chat endpoints are open.

启用认证时，聊天相关接口需要合法 token。

使用 `--disable-auth` 关闭认证后，注册/登录相关接口会返回 `503`，但聊天接口可匿名访问。

4.2 Error shape / 错误返回

Most errors use a minimal JSON shape:

大多数错误都使用一个最小 JSON 结构：

```
{
  "error": "message text"
}
```

Some endpoints may include extra debugging fields, for example:

```
{
  "error": "service config is fixed at startup; restart the API with --config to change it",
  "config_path": "F:\\AI\\M-Agent\\config\\agents\\chat\\chat_controller.yaml"
}
```

4.3 Public vs internal thread ids / 公共线程 ID 与内部线程 ID

When auth is enabled:

- request path / body still uses the public thread id, for example `demo-thread`
- the runtime internally uses `username::demo-thread`
- public API responses usually convert the thread id back to the public form

认证开启后，调用方仍使用公开线程 ID，但服务内部会自动做用户隔离。

5. Shared Schemas / 公共对象模型

This section describes the recurring objects used by multiple endpoints.

本节描述多个接口反复出现的对象结构。

5.1 `ErrorResponse`

Field	Type	中文说明	English description
`error`	`string`	错误信息	Error message

5.2 `AuthenticatedUser`

Field	Type	中文说明	English description
`username`	`string`	登录用户名，小写规范化	Login username, normalized to lowercase
`display_name`	`string`	展示名	Display name
`role`	`string`	当前角色， `basic` 或 `advanced`	Current role, `basic` or `advanced`
`config_path`	`string`	当前用户生效的 chat config 路径	Effective chat config path for this user
`created_at`	`string`	用户创建时间	User creation time
`updated_at`	`string`	用户最近配置更新时间	Last config update time
`editable_fields`	`object`	按配置 section 给出可编辑字段列表	Editable field names grouped by config section

5.3 `RunAcceptedResponse`

Returned by `POST /v1/chat/runs` .

Field	Type	中文说明	English description
`run_id`	`string`	异步 run ID	Asynchronous run id
`status`	`string`	初始状态，通常为 `queued`	Initial status, usually `queued`
`thread_id`	`string`	公开线程 ID	Public thread id
`user_id`	`string \	null`	归属用户；匿名模式下可能为空
`events_url`	`string`	该 run 的 SSE 地址	SSE URL for this run
`result_url`	`string`	该 run 的最终结果地址	Final result URL for this run

5.4 `RunSnapshot`

Returned by `GET /v1/chat/runs/{run_id}` .

Field	Type	中文说明	English description
`run_id`	`string`	run ID	Run id

Field	Type	中文说明	English description
`status`	`string`	`queued` / `running` / `completed` / `failed`	Run status
`config_path`	`string`	本 run 实际使用的 config 路径	Config path used by this run
`thread_id`	`string`	公开线程 ID	Public thread id
`user_id`	`string \	null`	归属用户
`message`	`string`	用户本轮输入	User message for this run
`created_at`	`string`	run 创建时间	Run creation time
`finished_at`	`string \	null`	run 完成时间
`event_count`	`integer`	当前已累计事件数	Number of captured events
`result`	`object \	null`	最终 chat result，对应下文 `ChatResult`
`error`	`string \	null`	失败时的错误文本

5.5 `ChatResult`

The `result` object inside a completed run snapshot.

`RunSnapshot.result` 中的最终业务结果对象。

Field	Type	中文说明	English description
`success`	`boolean`	本次 inbox drain 是否全部成功	Whether every processed stimulus succeeded
`thread_id`	`string`	公开线程 ID	Public thread id
`results`	`array[object]`	本次 drain 中每个刺激的事务结果	Per-stimulus transaction results from this drain
`replies`	`array[string]`	本次 drain 通过 `reply_to_user` 发出的回复	Replies emitted through `reply_to_user` during this drain
`answer`	`string`	`replies` 的最后一项；无回复时为空字符串	Last item in `replies`, or an empty string when no reply was emitted
`memory_write`	`null`	当前 run 返回中固定为 `null`	Currently always `null` in run output
`memory_capture`	`object`	当前轮的 memory capture 状态	Memory capture state for this round
`thread_state`	`object`	本轮结束后的线程状态快照	Thread-state snapshot after this run

`results` entries are scheduler outcomes, not a second answer schema. Depending on the stimulus and transaction phase, an entry may include fields such as `transaction_id`, `delegate_id`, `waiting_feedback`, `completed`, `silent`, `preempted`, `cancelled`, `summary`, or `error`.

`results` 中的条目是调度结果，不是另一套回答 schema。字段随刺激类型和事务阶段变化，常见字段包括 `transaction_id`、`delegate_id`、`waiting_feedback`、`completed`、`silent`、`preempted`、`cancelled`、`summary` 与 `error`。

5.6 `MemoryCapture`

Field	Type	中文说明	English description
`mode`	`string`	线程当时的 memory mode	Thread memory mode at capture time
`status`	`string`	常见值：`buffered`、`skipped`	Common values: `buffered`, `skipped`
`reason`	`string \	null`	跳过原因
`pending_rounds`	`integer`	当前 pending 轮次数	Number of pending rounds
`pending_turns`	`integer`	当前 pending turn 数	Number of pending turns

5.7 `ThreadState`

Field	Type	中文说明	English description
`thread_id`	`string`	公开线程 ID	Public thread id
`mode`	`string`	`manual` 或 `off`	`manual` or `off`
`history_rounds`	`integer`	当前线程保留的总轮次数	Total retained rounds
`history_messages`	`integer`	当前发给 chat history 的消息数	Current count of chat-history messages
`pending_rounds`	`integer`	尚未 flush 的轮次数	Pending round count
`pending_turns`	`integer`	尚未 flush 的 turn 数	Pending turn count
`has_pending_data`	`boolean`	是否存在待写回数据	Whether pending data exists
`last_activity_at`	`string`	最近一轮 assistant 完成时间	Last assistant activity time
`last_flush_at`	`string \	null`	最近一次成功 flush 时间
`last_flush_attempt_at`	`string \	null`	最近一次尝试 flush 时间
`last_flush_reason`	`string \	null`	最近一次 flush 的 reason
`last_flush_success`	`boolean \	null`	最近一次 flush 是否成功
`idle_flush_seconds`	`integer`	当前线程使用的空闲 flush 配置	Idle flush timeout

Field	Type	中文说明	English description
`idle_flush_deadline`	`string \	null`	如存在 pending 数据，则给出预计自动 flush 时间
`history_rounds_data`	`array[object]`	当前线程历史轮次明细	Detailed retained rounds
`history_preview`	`array[object]`	最近 3 条轮次预览	Last 3 retained rounds

Each item in `history_rounds_data` / `history_preview` contains:

- `round_id`
- `capture_state`
- `flush_id`
- `user_message`
- `assistant_message`
- `user_at`
- `assistant_at`

5.8 `DialogueSummary`

Returned inside `GET /v1/chat/dialogues`.

Field	Type	中文说明	English description
`dialogue_id`	`string`	对话归档 ID	Dialogue archive id
`thread_id`	`string`	公开线程 ID	Public thread id
`start_time`	`string \	null`	对话开始时间
`end_time`	`string \	null`	对话结束时间
`source`	`string \	null`	来源，例如 `chat_api_thread_flush`
`round_count`	`integer`	轮次数	Round count
`turn_count`	`integer`	turn 数	Turn count
`preview`	`string`	预览文本	Preview text
`dialogue_file`	`string`	后端文件路径，便于排查	Backend file path for debugging

5.9 `DialogueDetail`

Returned by `GET /v1/chat/dialogues/{dialogue_id}`.

Field	Type	中文说明	English description
`dialogue_id`	`string`	对话归档 ID	Dialogue id

Field	Type	中文说明	English description
`thread_id`	`string`	公开线程 ID	Public thread id
`thread_id_internal`	`string \	null`	内部线程 ID，调试字段
`user_id`	`string \	null`	对话用户标识
`participants`	`array[string]`	对话参与者	Dialogue participants
`meta`	`object`	原始元数据	Original metadata
`turns`	`array[object]`	标准化 turn 列表	Normalized turn list
`round_count`	`integer`	轮次数	Round count
`turn_count`	`integer`	turn 数	Turn count
`dialogue_file`	`string`	后端文件路径	Backend file path

Each `turns[]` item contains:

- `turn_id`
- `speaker`
- `text`
- `timestamp`

5.10 `ScheduleItem`

Field	Type	中文说明	English description
`schedule_id`	`string`	日程 ID，形如 `sch_xxx`	Schedule id, usually `sch_xxx`
`owner_id`	`string`	日程归属 owner	Schedule owner id
`thread_id`	`string`	该日程真正绑定的公开线程 ID	Actual public thread id bound to this schedule
`title`	`string`	日程标题	Schedule title
`status`	`string`	`pending` / `leased` / `running` / `done` / `failed` / `canceled`	Schedule status
`due_at_utc`	`string`	UTC 到期时间	UTC due time
`due_at_local`	`string`	本地时区时间	Local due time
`due_display`	`string`	适合 UI 显示的本地时间	UI-friendly local time
`timezone_name`	`string`	IANA 时区名	IANA timezone name
`original_time_text`	`string`	用户原始时间文本或标准化文本	Original or normalized time text

Field	Type	中文说明	English description
`action_type`	`string`	当前 API 创建的任务固定为 `chat_prompt`	Current API creates `chat_prompt` actions
`action_payload`	`object`	执行动作负载	Execution payload
`created_at`	`string`	创建时间	Creation time
`updated_at`	`string`	更新时间	Update time
`source_text`	`string`	来源文本	Source text
`metadata`	`object`	扩展元数据	Extra metadata

5.11 `ScheduleHeartbeat`

Field	Type	中文说明	English description
`enabled`	`boolean`	心跳是否启用	Whether heartbeat is enabled
`status`	`string`	`healthy` / `degraded` / `unhealthy`	Worker health summary
`worker.alive`	`boolean`	心跳线程是否存活	Whether the heartbeat worker is alive
`worker.created_at`	`string`	心跳协调器创建时间	Heartbeat coordinator creation time
`scheduler.beat_interval_seconds`	`integer`	扫描周期	Scan interval
`scheduler.interval_seconds`	`integer`	与 `beat_interval_seconds` 等价	Same as `beat_interval_seconds`
`scheduler.batch_limit`	`integer`	单次心跳最大 lease 数	Max leases per beat
`scheduler.enqueue_retry_seconds`	`integer`	入队失败后的 lease 重试延迟	Lease retry delay after an enqueue failure
`scheduler.next_beat_due_at`	`string \	null`	下一次扫描时间
`counters.beats_total`	`integer`	已执行心跳次数	Total beats executed
`counters.schedule_leased_total`	`integer`	已 lease 的任务总数	Total leased items
`counters.schedule_started_total`	`integer`	已入队的任务总数	Total enqueued items
`counters.schedule_completed_total`	`integer`	后台完成的任务总数	Total items completed in the background

Field	Type	中文说明	English description
`counters.schedule_failed_total`	`integer`	已失败任务总数	Total failed items
`last_beat.started_at`	`string \	null`	最近一次扫描开始时间
`last_beat.finished_at`	`string \	null`	最近一次扫描结束时间
`last_error`	`string \	null`	最近错误

5.12 `SSEEnvelope`

Every SSE event uses the standard envelope below:

所有 SSE 事件都遵循下面这个标准包裹层：

```
id: <seq>
event: <type>
data: {"run_id":"...","seq":1,"timestamp":"...","type":"run_started","payload":{...}}
```

Common fields:

Field	Type	中文说明	English description
`run_id` or `thread_id`	`string`	对应 run 或 thread 的标识	Run id or thread id
`seq`	`integer`	单流内单调递增序号	Monotonic sequence number within the stream
`timestamp`	`string`	事件生成时间	Event timestamp
`type`	`string`	事件类型	Event type
`payload`	`object`	事件主体	Event payload

6. Endpoint Reference / 端点说明

6.1 `GET /` and `GET /healthz`

用途 / Purpose:

- 返回服务健康信息、运行时配置概览、可用端点清单
- Return health info, runtime metadata, and endpoint map

Auth / 鉴权:

- none

Response fields:

Field	Type	中文说明	English description
`ok`	`boolean`	固定为 `true`	Always `true`
`service`	`string`	服务名，当前为 `m-agent-chat-api`	Service name
`root`	`string`	项目根目录	Project root path
`runtime`	`object`	主 chat runtime 健康信息	Main chat runtime health payload
`schedule_heartbeat`	`object`	日程心跳健康信息	Schedule heartbeat health payload
`auth`	`object \	null`	认证服务健康信息；匿名模式下为 `null`
`endpoints`	`object`	主要端点映射	Key endpoint map
`auth_required_for_chat`	`boolean`	聊天接口是否要求 token	Whether chat endpoints require auth

Example:

```
{
  "ok": true,
  "service": "m-agent-chat-api",
  "root": "F:\\AI\\M-Agent",
  "runtime": {
    "config_path": "F:\\AI\\M-Agent\\config\\agents\\chat\\chat_controller.yaml",
    "default_thread_id": "test-agent-1",
    "persist_memory": true,
    "idle_flush_seconds": 1800,
    "history_max_rounds": 12
  },
  "auth_required_for_chat": true
}
```

6.2 `POST /v1/auth/register`

用途 / Purpose:

- 注册用户并生成其专属配置目录
- Register a user and scaffold a user-specific config bundle

Auth / 鉴权:

- none
- returns `503` if auth is disabled

Request body:

Field	Type	Required	中文说明	English description
`username`	`string`	yes	用户名，3-32 位，只允许字母、数字、点、下划线、横线	Username, 3-32 chars, letters/digits/dot/underscore/dash only
`password`	`string`	yes	密码，至少 8 位	Password, at least 8 characters
`role`	`string`	no	`basic` 或 `advanced` ，默认 `basic`	`basic` or `advanced` , default `basic`
`display_name`	`string`	no	展示名	Display name
`assistant_name`	`string`	no	聊天助手显示名	Chat assistant display name
`persona_prompt`	`string`	no	用户自定义 persona prompt	Custom persona prompt
`workflow_id`	`string`	no	记忆工作流隔离 ID	Memory workflow namespace

Success response:

```
{
  "user": {
    "username": "alice",
    "display_name": "alice",
    "role": "basic",
    "config_path": "F:\\AI\\M-Agent\\config\\users\\alice\\chat.yaml",
    "created_at": "2026-04-05T09:00:00Z",
    "updated_at": "2026-04-05T09:00:00Z",
    "editable_fields": {
      "chat": ["chat_assistant_name", "chat_persona_prompt"],
      "memory_agent": [],
      "memory_core": []
    }
  }
}
```

Common errors:

- `400` : invalid username, short password, invalid role, or missing fields
- `409` : user already exists
- `503` : auth service disabled

6.3 `POST /v1/auth/login`

用途 / Purpose:

- 用户登录并获取 Bearer token

- Login and obtain a Bearer token

Auth / 鉴权:

- none
- returns `503` if auth is disabled

Request body:

Field	Type	Required	中文说明	English description
`username`	`string`	yes	用户名	Username
`password`	`string`	yes	密码	Password

Success response:

Field	Type	中文说明	English description
`user`	`AuthenticatedUser`	用户信息	User info
`access_token`	`string`	Bearer token	Bearer token
`token_type`	`string`	固定为 `bearer`	Always `bearer`
`expires_at`	`string`	token 过期时间	Token expiration time

Example:

```
{
  "user": {
    "username": "alice",
    "display_name": "alice",
    "role": "basic",
    "config_path": "F:\\AI\\M-Agent\\config\\users\\alice\\chat.yaml",
    "created_at": "2026-04-05T09:00:00Z",
    "updated_at": "2026-04-05T09:00:00Z",
    "editable_fields": {
      "chat": ["chat_assistant_name", "chat_persona_prompt"],
      "memory_agent": [],
      "memory_core": []
    }
  },
  "access_token": "paste-me",
  "token_type": "bearer",
  "expires_at": "2026-04-05T21:00:00Z"
}
```

Common errors:

- `400` : missing username or password
- `401` : invalid username or password

- `503` : auth service disabled

6.4 `GET /v1/auth/me`

用途 / Purpose:

- 返回当前 token 对应的用户信息
- Return the current authenticated user

Auth / 鉴权:

- required when auth is enabled

Success response:

```
{
  "user": {
    "username": "alice",
    "display_name": "alice",
    "role": "basic",
    "config_path": "F:\\AI\\M-Agent\\config\\users\\alice\\chat.yaml",
    "created_at": "2026-04-05T09:00:00Z",
    "updated_at": "2026-04-05T09:00:00Z",
    "editable_fields": {
      "chat": ["chat_assistant_name", "chat_persona_prompt"],
      "memory_agent": [],
      "memory_core": []
    }
  }
}
```

Common errors:

- `401` : missing/invalid/expired token
- `503` : auth service disabled

6.5 `POST /v1/auth/logout`

用途 / Purpose:

- 使当前 token 失效
- Invalidate the current token

Auth / 鉴权:

- required when auth is enabled

Success response:

```
{
  "success": true
```

```
}
```

6.6 `GET /v1/users/me/config/schema`

用途 / Purpose:

- 返回当前用户可查看/可编辑的配置 schema 与当前值
- Return editable config schema metadata and current values for the current user

Auth / 鉴权:

- required

Top-level response:

Field	Type	中文说明	English description
`user`	`object`	当前用户与配置路径信息	Current user and config path
`sections`	`object`	`chat` / `memory_agent` / `memory_core` 三个 section	Three config sections

Each `sections.<name>` object contains:

Field	Type	中文说明	English description
`editable_fields`	`array[string]`	当前角色可修改字段列表	Fields editable for the current role
`fields`	`object`	字段元数据映射	Field metadata map
`patch_example`	`object`	当前值裁出来的 patch 示例	Patch example built from current values

Each `fields.<key>` object contains:

Field	Type	中文说明	English description
`type`	`string`	字段类型说明	Field type
`description`	`string`	字段用途说明	Field description
`editable`	`boolean`	当前角色能否改这个字段	Whether the current role may edit the field
`present`	`boolean`	当前配置里是否出现该字段	Whether the field is present in current config
`current_value`	`any`	当前值	Current value

6.7 `PATCH /v1/users/me/config`

用途 / Purpose:

- 更新当前用户的可编辑配置
- Update editable config values for the current user

Auth / 鉴权:

- required

Request body:

```
{
  "chat": {
    "chat_assistant_name": "Memory Assistant",
    "chat_persona_prompt": "Be concise."
  },
  "memory_agent": {},
  "memory_core": {}
}
```

Body rules / 请求规则:

- top-level keys are `chat`, `memory_agent`, `memory_core`
- each section must be an object if present
- at least one effective field change is required
- unsupported keys return `400`
- disallowed keys for the current role return `403`

Current editable field matrix:

Role `basic`:

- `chat.chat_assistant_name`
- `chat.chat_persona_prompt`

Role `advanced` adds:

- `chat.chat_user_name`
- `chat.persist_memory`
- `chat.enabled_tools`
- `chat.tool_defaults`
- `chat.thread_id`
- `memory_agent.model_name`
- `memory_agent.agent_temperature`
- `memory_agent.recursion_limit`
- `memory_agent.retry_recursion_limit`
- `memory_agent.detail_search_defaults`
- `memory_agent.network_retry_attempts`
- `memory_agent.network_retry_backoff_seconds`
- `memory_agent.network_retry_backoff_multiplier`
- `memory_agent.network_retry_max_backoff_seconds`

- `memory_core.workflow_id`
- `memory_core.memory_owner_name`
- `memory_core.memory_similarity_threshold`
- `memory_core.memory_top_k`
- `memory_core.memory_use_threshold`
- `memory_core.embed_provider`

Success response:


```
{
  "user": {
    "username": "alice",
    "display_name": "alice",
    "role": "basic",
    "config_path": "F:\\AI\\M-Agent\\config\\users\\alice\\chat.yaml",
    "created_at": "2026-04-05T09:00:00Z",
    "updated_at": "2026-04-05T09:10:00Z",
    "editable_fields": {
      "chat": ["chat_assistant_name", "chat_persona_prompt"],
      "memory_agent": [],
      "memory_core": []
    }
  }
}
```


6.8 `POST /v1/chat/runs`

用途 / Purpose:

- 创建一个异步 chat run
- Create an asynchronous chat run

Auth / 鉴权:

- required when auth is enabled
- open in anonymous mode

Request body:

Field	Type	Required	中文说明	English description
`thread_id`	`string`	no	线程 ID；为空时使用默认线程	Thread id; defaults to runtime default thread
`message`	`string`	yes	用户消息	User message
`config`	`string`	no	当前实现不支持，请勿传	Not supported in current implementation

Behavior notes / 行为说明:

- if `config` is provided and non-empty, the server returns `400`
- successful creation returns only run metadata, not the final answer

Example request:

```
{
  "thread_id": "demo-thread",
  "message": "帮我回忆一下我上次提到的旅行计划。"
}
```

Success response:

```
{
  "run_id": "run_123",
  "status": "queued",
  "thread_id": "demo-thread",
  "user_id": "alice",
  "events_url": "/v1/chat/runs/run_123/events",
  "result_url": "/v1/chat/runs/run_123"
}
```

Common errors:

- `400` : message empty or unsupported request-level config override
- `401` : missing/invalid token when auth is enabled

6.9 `GET /v1/chat/runs/{run_id}`

用途 / Purpose:

- 获取某个 run 的最终快照
- Fetch the snapshot of a run

Auth / 鉴权:

- same auth visibility as the run owner

Notes / 说明:

- if auth is enabled, users can only read their own runs
- `result` becomes non-null only after completion

6.10 `GET /v1/chat/runs/{run_id}/events`

用途 / Purpose:

- 订阅单个 run 的完整实时事件流
- Subscribe to the full real-time event stream of a single run

Auth / 鉴权:

- same auth visibility as the run owner

Query params:

Param	Type	Default	中文说明	English description
`after_seq`	`integer`	`0`	仅返回 `seq > after_seq` 的事件	Return only events with `seq > after_seq`

Stream behavior / 事件流行为:

- media type is `text/event-stream`
- emits `: keep-alive` comments when idle
- closes automatically after the run is done and no further tail events remain

Recommended usage / 推荐用法:

1. call `POST /v1/chat/runs`
2. immediately connect to the run SSE URL
3. optionally call `GET /v1/chat/runs/{run_id}` for final verification

6.11 `GET /v1/chat/threads/{thread_id}/events`

用途 / Purpose:

- 订阅线程级生命周期事件
- Subscribe to thread lifecycle events

Auth / 鉴权:

- same auth visibility as the thread owner

Query params:

Param	Type	Default	中文说明	English description
`after_seq`	`integer`	`-1`	默认从当前尾部开始，仅看新事件；传 `0` 可回放当前缓存	Default follows only new events from the live tail; pass `0` to replay current buffer

Important note / 重要说明:

- this is not the same as the run trace stream
- do not use it as a replacement for `/v1/chat/runs/{run_id}/events`

当前实现里，thread 事件流主要包含：

- memory mode changes
- memory flush progress
- schedule CRUD events
- schedule execution events

6.12 `GET /v1/chat/threads/{thread_id}/memory/state`

用途 / Purpose:

- 获取线程当前内存状态快照
- Fetch the current thread memory snapshot

Auth / 鉴权:

- same auth visibility as the thread owner

Success response:

- `ThreadState`

6.13 `POST /v1/chat/threads/{thread_id}/memory/mode`

用途 / Purpose:

- 切换线程 memory mode
- Change the thread memory mode

Auth / 鉴权:

- same auth visibility as the thread owner

Request body:

Field	Type	Required	中文说明	English description
`mode`	`string`	no	目标模式，支持 `manual` 和 `off`	Target mode, `manual` or `off`
`discard_pending`	`boolean`	no	是否把当前 pending 轮次标记为 `skipped`	Whether to mark current pending rounds as `skipped`

Behavior note / 行为说明:

- invalid or empty `mode` falls back to `manual`
- `discard_pending` does not delete history; it only marks pending rounds as skipped

Success response fields:

Field	Type	中文说明	English description
`success`	`boolean`	是否成功	Whether the change succeeded
`thread_id`	`string`	公开线程 ID	Public thread id
`mode`	`string`	生效模式	Effective mode
`discard_pending`	`boolean`	是否执行了 pending 丢弃逻辑	Whether pending discard logic was requested
`thread_state`	`ThreadState`	更新后的线程状态	Updated thread state

6.14 `POST /v1/chat/threads/{thread_id}/memory/flush`

用途 / Purpose:

- 手动把 pending 对话轮次写入长期记忆
- Manually flush pending chat rounds into long-term memory

Auth / 鉴权:

- same auth visibility as the thread owner

Request body:

Field	Type	Required	中文说明	English description
`reason`	`string`	no	flush 原因，默认 `manual_api`	Flush reason, default `manual_api`

Success response fields:

Field	Type	中文说明	English description
`success`	`boolean`	flush 是否成功	Whether flush succeeded
`thread_id`	`string`	公开线程 ID	Public thread id
`flush_reason`	`string`	flush reason	Flush reason
`status`	`string`	`noop` / `written` / `failed`	Flush status
`message`	`string \	null`	无待写回时的提示文本
`rounds_flushed`	`integer`	本次写入轮次数	Number of flushed rounds
`turns_flushed`	`integer`	本次写入 turn 数	Number of flushed turns
`memory_write`	`object \	null`	记忆写入结果摘要
`thread_state`	`ThreadState`	flush 后线程状态	Thread state after flush
`error`	`string \	null`	失败错误

Notes / 说明:

- if there are no pending rounds, the endpoint returns `success: true` and `status: "noop"`
- flush progress is also emitted to the thread SSE stream

6.15 `GET /v1/chat/dialogues`

用途 / Purpose:

- 获取已 flush 到归档目录的对话列表
- List archived dialogues already flushed to storage

Auth / 鉴权:

- same auth visibility as the current user

Query params:

Param	Type	Default	中文说明	English description
`thread_id`	`string`	empty	按线程过滤	Filter by thread
`limit`	`integer`	`30`	返回条数	Page size
`offset`	`integer`	`0`	偏移量	Offset

Response fields:

Field	Type	中文说明	English description
`items`	`array[DialogueSummary]`	当前页归档项	Current page of dialogue summaries
`offset`	`integer`	当前 offset	Current offset
`limit`	`integer`	当前 limit	Current limit
`next_offset`	`integer \	null`	下一页 offset
`has_more`	`boolean`	是否还有下一页	Whether more pages exist
`total`	`integer`	总数	Total count

6.16 `GET /v1/chat/dialogues/{dialogue_id}`

用途 / Purpose:

- 获取某个归档对话的详细内容
- Fetch the details of one archived dialogue

Auth / 鉴权:

- same auth visibility as the current user

Success response:

- `DialogueDetail`

Common errors:

- `404` : not found or not visible to the current user

6.17 `GET /v1/chat/threads/{thread_id}/schedules`

用途 / Purpose:

- 列出当前 owner 的日程
- List schedules for the current owner

Auth / 鉴权:

- same auth visibility as the current user

Query params:

Param	Type	Default	中文说明	English description
`include_completed`	`boolean`	`false`	是否包含已结束状态	Include terminal statuses
`limit`	`integer`	`20`	返回条数，服务端会限制到 `1..100`	Page size, clamped to `1..100`
`keyword`	`string`	empty	关键字搜索	Keyword search
`statuses`	`string`	empty	逗号分隔的状态列表	Comma-separated status list

Important note / 重要说明:

- this endpoint is owner-scoped, not strictly thread-scoped
- the path `thread_id` is mainly used as a public wrapper value
- list results may include schedules whose actual `item.thread_id` belongs to another thread of the same owner

这个行为是当前实现的真实语义，测试时不要误以为列表一定只包含路径里的那个线程。

Success response fields:

Field	Type	中文说明	English description
`thread_id`	`string`	请求路径中的公开线程 ID	Public thread id from the request path
`scope`	`string`	固定为 `owner`	Always `owner`
`owner_id`	`string`	当前 owner	Current owner id
`count`	`integer`	返回项数	Returned item count
`include_completed`	`boolean`	是否包含终态	Whether terminal items are included
`keyword`	`string`	生效关键字	Effective keyword
`statuses`	`array[string]`	生效状态过滤器	Effective status filter
`items`	`array[ScheduleItem]`	日程列表	Schedule items
`heartbeat`	`ScheduleHeartbeat`	当前心跳状态摘要	Current heartbeat summary

6.18 `GET /v1/chat/threads/{thread_id}/schedules/heartbeat`

用途 / Purpose:

- 返回日程心跳工作器状态

- Return schedule heartbeat worker status

Auth / 鉴权:

- same auth visibility as the current user

Success response:


```
{
  "thread_id": "demo-thread",
  "scope": "owner",
  "heartbeat": {
    "enabled": true,
    "worker_alive": true,
    "beat_interval_seconds": 10
  }
}
```


6.19 `GET /v1/chat/threads/{thread_id}/schedules/{schedule_id}`

用途 / Purpose:

- 获取某个日程详情
- Fetch one schedule item

Auth / 鉴权:

- same auth visibility as the current user

Important note / 重要说明:

- lookup is owner-scoped by `schedule_id`
- the wrapper `thread_id` comes from the request path
- the actual bound thread is `item.thread_id`

Success response:


```
{
  "thread_id": "demo-thread",
  "item": {
    "schedule_id": "sch_abc123",
    "thread_id": "work-thread",
    "title": "Weekly report",
    "status": "pending"
  }
}
```


6.20 `POST /v1/chat/threads/{thread_id}/schedules`

用途 / Purpose:

- 创建一个聊天提醒型日程
- Create a chat-prompt schedule

Auth / 鉴权:

- same auth visibility as the current user

Request body:

Field	Type	Required	中文说明	English description
`title`	`string`	conditional	标题；与 `prompt` 至少二选一	Title; required unless `prompt` or `source_text` is present
`prompt`	`string`	conditional	到点时发给 chat 的提示文本	Prompt sent to chat when due
`due_at`	`string`	yes	ISO datetime 字符串	ISO datetime string
`timezone_name`	`string`	no	时区名；无 offset 时间会按该时区解释	Timezone name used when `due_at` has no offset
`original_time_text`	`string`	no	原始时间文本	Original time text
`source_text`	`string`	no	来源文本	Source text
`metadata`	`object`	no	扩展元数据	Extra metadata

Rules / 规则:

- if both `title` and `prompt` are empty, the server also tries `source_text`
- `due_at` must be a valid ISO datetime string
- if `due_at` has no timezone offset, the server applies `timezone_name`
- the created `action_type` is fixed as `chat_prompt`

Success response:

```
{
  "success": true,
  "thread_id": "demo-thread",
  "item": {
    "schedule_id": "sch_abc123",
    "thread_id": "demo-thread",
    "title": "交周报",
    "status": "pending",
    "due_at_utc": "2026-04-06T01:30:00Z",
    "due_at_local": "2026-04-06T09:30:00+08:00",
    "due_display": "2026-04-06 09:30",
    "timezone_name": "Asia/Shanghai"
```

```
  }
}
```

6.21 `PATCH /v1/chat/threads/{thread_id}/schedules/{schedule_id}`

用途 / Purpose:

- 更新日程字段
- Update schedule fields

Auth / 鉴权:

- same auth visibility as the current user

Request body:

All fields are optional.

所有字段都可选。

Field	Type	中文说明	English description
`title`	`string`	新标题	New title
`prompt`	`string`	更新 action payload 里的 prompt	Update the prompt inside action payload
`due_at`	`string`	新到期时间	New due datetime
`timezone_name`	`string`	新时区	New timezone
`original_time_text`	`string`	新的原始时间文本	New original time text
`source_text`	`string`	新来源文本	New source text
`metadata`	`object`	要 merge 的 metadata	Metadata patch to merge

Rules / 规则:

- if `prompt` is present, it cannot be empty
- if `due_at` is updated, the server normalizes timezone fields again
- update is owner-scoped by `schedule_id`

6.22 `DELETE /v1/chat/threads/{thread_id}/schedules/{schedule_id}`

用途 / Purpose:

- 取消日程
- Cancel a schedule

Auth / 鉴权:

- same auth visibility as the current user

Behavior / 行为:

- the item status becomes `canceled`
- `metadata.canceled_at` is recorded
- cancellation is owner-scoped by `schedule_id`

Success response:

```
{
  "success": true,
  "thread_id": "demo-thread",
  "item": {
    "schedule_id": "sch_abc123",
    "thread_id": "work-thread",
    "status": "canceled"
  }
}
```

7. SSE Reference / SSE 事件参考

7.1 Run stream events / Run 级事件

The following event types may appear on `GET /v1/chat/runs/{run_id}/events`.

下列事件类型可能出现在 `GET /v1/chat/runs/{run_id}/events`。

Event type	中文说明	English description	Common payload fields
`run_started`	run 开始	Run started	`thread_id`, `message`, `config_path`, `user_id`
`recall_started`	recall 工具开始	Recall started	`mode`, `question`
`tool_call`	工具调用开始	Tool call started	`call_id`, `tool_name`, `status`, `params`, optional `ts`
`tool_result`	工具调用完成或失败	Tool call completed or failed	`call_id`, `tool_name`, `status`, optional `result`, optional `error`
`recall_completed`	recall 结束	Recall completed	`mode`, `question`, `answer`
`assistant_message`	面向用户的最终回答	Final user-facing answer	`thread_id`, `answer`
`memory_capture_updated`	当前轮次的 memory capture 状态	Memory capture status for the run	`mode`, `status`, `reason`, `pending_rounds`, `pending_turns`
`thread_state_updated`	当前 run 看到的线程状态快照	Thread-state snapshot observed by the run	`thread_state`

Event type	中文说明	English description	Common payload fields
`run_completed`	run 成功结束	Run completed	`thread_id` , `answer` , `result`
`run_failed`	run 失败	Run failed	`thread_id` , `error`

Important notes / 重要说明:

- `run_completed.payload.result` already contains the final business result, so many clients do not need an extra `GET /v1/chat/runs/{run_id}` call
- `tool_call` and `tool_result` are live traces from direct capability invocation; planning events are published on the thread stream

7.2 Thread stream events / Thread 级事件

The following event types may appear on `GET /v1/chat/threads/{thread_id}/events` .

下列事件类型可能出现在 `GET /v1/chat/threads/{thread_id}/events` 。

Event type	中文说明	English description	Common payload fields
`thread_state_updated`	线程状态变化	Thread-state change	`thread_state`
`flush_started`	手动或空闲 flush 开始	Flush started	`operation_id` , `thread_id` , `flush_reason` , `pending_rounds` , `pending_turns`
`flush_stage`	flush 各阶段进度	Flush stage progress	`operation_id` , `thread_id` , `flush_reason` , `stage` , `stage_label` , `status` , optional `result` , optional `error`
`flush_completed`	flush 完成	Flush completed	`operation_id` , `thread_id` , `flush_reason` , `success` , `status` , `rounds_flushed` , `turns_flushed` , optional `memory_write` , optional `error` , `thread_state`
`schedule_created`	手动创建日程	Schedule created	`thread_id` , `schedule`
`schedule_updated`	手动更新日程	Schedule updated	`thread_id` , `schedule`
`schedule_canceled`	手动取消日程	Schedule canceled	`thread_id` , `schedule`
`schedule_due`	心跳发现到点任务并入队	A due schedule was leased and enqueued	`thread_id` , `schedule_id` , `title` , `status` , `due_at_utc` , `timezone_name`

Event type	中文说明	English description	Common payload fields
`schedule_queued`	到点任务进入 Think-life inbox	Due schedule enqueued in the Think-life inbox	`thread_id` , `schedule_id` , `run_id` , `stimulus_id` , `pending_count` , `runtime_phase`
`schedule_started`	Think-life 开始消费日程刺激	Think-life started consuming the schedule stimulus	`thread_id` , `schedule_id` , `run_id`
`schedule_completed`	Think-life 日程事务完成	Think-life schedule transaction completed	`thread_id` , `schedule_id` , `run_id` , `status` , `answer`
`schedule_failed`	Think-life 日程事务失败	Think-life schedule transaction failed	`thread_id` , `schedule_id` , `run_id` , `error`
`stimulus_queued`	刺激入队	Stimulus enqueued	`stimulus_id` , `kind` , `pending_count`
`reply_emitted`	用户可见回复	User-visible reply	`message` , `finalize` , `transaction_id` , `delegate_id`
`scene_entry_appended`	Scene 时间轴新增条目	Scene timeline entry appended	`seq` , `occurred_at` , `entry_type` , `actor` , `text` , `transaction_id`
`thread_runtime_updated`	单线程队列快照	Per-thread queue snapshot	`thread_runtime`
`thinking_*`	思考层规划事件	Thinking-layer planning events	varies

Notes / 说明:

- thread streams do not automatically terminate like run streams
- `thread_id` inside thread SSE payload is converted back to the public thread id

8. Testing Recommendations / 测试建议

8.1 Minimal real-time chat flow / 最小实时聊天联调流程

1. Login if auth is enabled.
2. Call `POST /v1/chat/runs` .
3. Connect to `GET /v1/chat/runs/{run_id}/events` .
4. Render `assistant_message` and/or `run_completed.payload.result.answer` .
5. Optionally fetch `GET /v1/chat/runs/{run_id}` as a final snapshot.

8.2 Thread memory verification / 线程记忆验证

1. Run several chat turns on the same `thread_id` .
2. Call `GET /v1/chat/threads/{thread_id}/memory/state` .
3. Verify `pending_rounds` , `history_rounds` , and `idle_flush_deadline` .
4. Call `POST /v1/chat/threads/{thread_id}/memory/flush` .
5. Verify `status` , `memory_write` , and `pending_rounds == 0` .

8.3 Schedule verification / 日程验证

1. Create a schedule with `POST /v1/chat/threads/{thread_id}/schedules`.
2. Verify it appears in `GET /v1/chat/threads/{thread_id}/schedules`.
3. Subscribe to `GET /v1/chat/threads/{thread_id}/events`.
4. Wait for `schedule_due`, `schedule_started`, and `schedule_completed`.
5. Verify the schedule status becomes `done`.

8.4 Recommended companion file / 推荐配套文件

Use the request collection in:

请配合下面这个请求集合文件使用：

- `docs/chat_api/testing.http`

It contains ready-to-edit requests for:

- health
- register / login / me / logout
- config schema and patch
- create run / get run
- memory state / mode / flush
- dialogue list / detail
- schedule list / create / update / cancel

9. Change Notes / 变更说明

Compared with the old `docs/chat_api.md`, the new reference explicitly documents:

相较旧版 `docs/chat_api.md`，这份新文档明确补齐了这些当前实现细节：

- auth endpoints and auth-disabled behavior
- per-user runtime scoping
- user config schema and patch semantics
- dialogue archive APIs
- thread events versus run events
- schedule heartbeat and owner-scoped schedule behavior
- current request/response shapes from the actual implementation

Appendix: testing.http

This appendix includes the ready-to-edit request collection from `docs/chat_api/testing.http`.

```
@baseUrl = http://127.0.0.1:8777
@username = demo_user
@password = password123
@accessToken = paste_access_token_here
@threadId = demo-thread
@runId = run_xxx
@dialogueId = chat_xxx
@scheduleId = sch_xxx

### Health
GET {{baseUrl}}/healthz

### Auth
### Skip this group when the server runs with --disable-auth.

POST {{baseUrl}}/v1/auth/register
Content-Type: application/json

{
  "username": "{{username}}",
  "password": "{{password}}",
  "role": "basic",
  "display_name": "Demo User",
  "assistant_name": "Memory Assistant",
  "persona_prompt": "Be concise and helpful."
}

###

POST {{baseUrl}}/v1/auth/login
Content-Type: application/json

{
  "username": "{{username}}",
  "password": "{{password}}"
}

###

GET {{baseUrl}}/v1/auth/me
Authorization: Bearer {{accessToken}}

###

POST {{baseUrl}}/v1/auth/logout
Authorization: Bearer {{accessToken}}
```

```
### User config

GET {{baseUrl}}/v1/users/me/config/schema
Authorization: Bearer {{accessToken}}

###

PATCH {{baseUrl}}/v1/users/me/config
Authorization: Bearer {{accessToken}}
Content-Type: application/json

{
  "chat": {
    "chat_assistant_name": "Memory Assistant",
    "chat_persona_prompt": "Please answer in concise Chinese."
  }
}

### Chat run

POST {{baseUrl}}/v1/chat/runs
Authorization: Bearer {{accessToken}}
Content-Type: application/json

{
  "thread_id": "{{threadId}}",
  "message": "Please summarize the main points we discussed before."
}

###

GET {{baseUrl}}/v1/chat/runs/{{runId}}
Authorization: Bearer {{accessToken}}

### SSE note
### Open in browser, curl, or an SSE client.
### curl -N -H "Authorization: Bearer <token>" "{{baseUrl}}/v1/chat/runs/{{runId}}/events"

GET {{baseUrl}}/v1/chat/runs/{{runId}}/events
Authorization: Bearer {{accessToken}}

### Thread events
### This stream is for memory flush and schedule lifecycle events.
### curl -N -H "Authorization: Bearer <token>" "{{baseUrl}}/v1/chat/threads/{{threadId}}/events?
after_seq=0"

GET {{baseUrl}}/v1/chat/threads/{{threadId}}/events?after_seq=0
Authorization: Bearer {{accessToken}}

### Thread memory

GET {{baseUrl}}/v1/chat/threads/{{threadId}}/memory/state
Authorization: Bearer {{accessToken}}
```

```
###

POST {{baseUrl}}/v1/chat/threads/{{threadId}}/memory/mode
Authorization: Bearer {{accessToken}}
Content-Type: application/json

{
  "mode": "manual",
  "discard_pending": false
}

###

POST {{baseUrl}}/v1/chat/threads/{{threadId}}/memory/flush
Authorization: Bearer {{accessToken}}
Content-Type: application/json

{
  "reason": "manual_test"
}

### Dialogues

GET {{baseUrl}}/v1/chat/dialogues?thread_id={{threadId}}&limit=20&offset=0
Authorization: Bearer {{accessToken}}

###

GET {{baseUrl}}/v1/chat/dialogues/{{dialogueId}}
Authorization: Bearer {{accessToken}}

### Schedules

GET {{baseUrl}}/v1/chat/threads/{{threadId}}/schedules
Authorization: Bearer {{accessToken}}

###

GET {{baseUrl}}/v1/chat/threads/{{threadId}}/schedules?
include_completed=true&limit=20&keyword=&statuses=
Authorization: Bearer {{accessToken}}

###

GET {{baseUrl}}/v1/chat/threads/{{threadId}}/schedules/heartbeat
Authorization: Bearer {{accessToken}}

###

POST {{baseUrl}}/v1/chat/threads/{{threadId}}/schedules
Authorization: Bearer {{accessToken}}
Content-Type: application/json
```

```
{
  "title": "Weekly report",
  "prompt": "Remind me to submit the weekly report",
  "due_at": "2026-04-06T09:30",
  "timezone_name": "Asia/Shanghai",
  "source_text": "manual testing create"
}

###

GET {{baseUrl}}/v1/chat/threads/{{threadId}}/schedules/{{scheduleId}}
Authorization: Bearer {{accessToken}}

###

PATCH {{baseUrl}}/v1/chat/threads/{{threadId}}/schedules/{{scheduleId}}
Authorization: Bearer {{accessToken}}
Content-Type: application/json

{
  "title": "Project weekly report",
  "prompt": "Remind me to submit the project weekly report",
  "due_at": "2026-04-06T10:15",
  "timezone_name": "Asia/Shanghai",
  "source_text": "manual testing update",
  "metadata": {
    "updated_by": "testing.http"
  }
}

###

DELETE {{baseUrl}}/v1/chat/threads/{{threadId}}/schedules/{{scheduleId}}
Authorization: Bearer {{accessToken}}
```